

Choose a substantial problem

September 30, 2026. **Unpiloted.** This revision applies the current Week 2 catalogs and AGENTS.md: short shared rules, several contrasting examples under one question, and methods left for children to choose. The pages are a choice library for an hour, not a completion assignment.

Prepare for ten children and three adults

Current roster: KK1 / 3333 / 445. Keep one adult at each table: the parent with the three youngest, the other mathematician with the four third graders, and the organizer with the three oldest. Offer pages across these groups when useful. Print three K first pages, four middle first pages, and three upper first pages: **ten starting sheets**. Bring a master of each continuation and this guide for the adults; children can continue on blank paper or existing whiteboards. No classroom printer is assumed. Paper, pencils/erasers, whiteboards and counters suffice. Optional hand-drawn cards make the action tangible: two sets of three shape cards per K child (18 cards total); a set of A–D for each middle child (16); A–E for each upper child (15), with F–I in reserve. Repeated symbols can be drawn on spare scraps. Keep an input row visible while building a separate output row.

Brief whole-group launch, then two work periods

0–4 minutes: let children make messages with the materials. **4–8:** show the shape key on K page 1. Send circle–triangle–circle through it, making triangle–square–triangle below it. One turn changes each old symbol once; it does not repeatedly change a newly written symbol. Let a child make the next turn. Show that an empty row before an arrow asks for the input. On a small letter-key example, show that a column pairs one input with one output. Then distribute pages; do not demonstrate a solving strategy. **8–28:** one chosen investigation. **28–33:** stand, stretch, reset. **33–50:** continue or offer another question. **50–55:** share an attempt or discovery. **55–60:** tidy. These are adjustable planning intervals. An absorbing problem can take both work periods.

Prerequisites and useful first offers

K, Problems 1–2: match shapes; adult reads; no arithmetic. Problem 3 adds counting to three and tracking a whole message. Problems 4–5 concern recoverable information.

Middle, 1: A–D are labels, not words; coordinate equal symbols and a one-to-one rule. Problems 2–5 add counting to six and comparing constructions.

Upper, 1–2: track five letters, count to six, and eventually justify that all cases are covered. Problems 3–4 are an alternative direction in composition.

Extra: common returns through 20 and organizing a completeness argument; no algebra notation or factorial prerequisite. Offer after a child understands whole-key return. A message, a useful impossible clue, or the five-versus-six-turn comparison can be a complete session. Allow roughly 8–15 minutes per ordinary investigation, and 15–30 or more for a maximum or completeness proof. Adults can read and scribe. Finishing one page is not a gate to another.

Shapes: messages and information

K 1–2: send, recover, invent

Read each arrow as one turn. The six missing rows in Problem 1, from top to bottom, are:

Row	Missing message
1	triangle–square–triangle
2	circle–circle–square
3	square–triangle–circle
4	circle–square–circle
5	triangle–triangle–square
6	square–circle–triangle

The first three rows ask for outputs, the last three for inputs. Repeated symbols must be treated consistently. Problem 2 sustains the same mathematics through partner invention: send a message, pass only the output, and recover the original using the printed key. Rotate sender, receiver, and checker at a three-child table. There is no required message length. **If needed:** cover all but one column of a message and ask which shape produces the visible output. Ask a child to replay a disputed turn. **Later question:** can the receiver ever have two possible originals with this key? Each output has exactly one input, so no. This explanation can use pointing alone.

K 3: whose return are we counting?

The three starts are square; circle–square–circle; circle–triangle–square. Under P, each returns first at turn **3**. Under Q, their returns are **1, 2, 2**. A return is after at least one turn; an unchanged square returns in one, not zero. Allow children to choose cards, drawings, or a list. Do not require six copied trace tables. If a child has lost the starting message, keep a separate copy. If counting one shape hides the rest, ask whether the *whole* message matches its start. To introduce loops later, replay circle’s three turns, placing arrows between the visited shapes; only then show that a counter can follow the same route. Let the child try that record from triangle and square. **Extension:** make a message that returns after one turn under Q, and one that takes two. Exactly the nonempty messages made entirely of squares have first return one. The others take two. Under P every nonempty message takes three, because no shape is fixed by one or two turns.

K 4: recoverable keys

A key must have one output for each input. It lets a partner *always* recover the message precisely when the three outputs are all different. The six possible output rows, in input order circle/triangle/square, are the six arrangements of these shapes. Identity is allowed; it returns in one turn. Three keys swap a pair and fix the third (return two); two cycle all three (return three). Children need not find all six. If they want completeness, choose the circle’s output in three ways, then the triangle’s in two, leaving one square output. If a child reuses an output, test two different one-shape messages. This reveals the issue through action instead of a premature prohibition. Optional extension: find every recoverable key and compare return times.

K 5: a key that loses information

Both circle and triangle become square; square becomes circle. The four inputs for square–square are circle–circle, circle–triangle, triangle–circle, and triangle–triangle. Each position has two choices, independently; no input square is possible. If needed, fix the first shape while changing the second. Extension: three output squares have eight possible inputs. A receiver cannot select one original without additional information.

Four letters: hidden keys and returns

Middle 1: every key, or none

Output rows below are in input order A,B,C,D. Each clue is a separate puzzle.

Clue	All possible output rows	Reason
a	CDAB, CDBA	A and B fixed by the clue; two outputs remain
b	BCDA	A,B,C fixed; one output remains
c	none	The two A's would have different outputs
d	none	A and B would share output C
e	ABCD, ABDC	A,B fixed; C,D can stay or swap
f	DBAC	A,B,C fixed; one output remains

The blank table holds a candidate; extra keys can go below or on a whiteboard. Clue c breaks consistency: identical inputs need identical outputs. Clue d breaks reversibility: distinct inputs need distinct outputs. If needed, ask which letter causes a conflict. Extension: invent clues with one, two, or no keys.

Middle 2-3: incomplete messages can hide the key

Starting message	P	Q	R
D	1	2	4
ABBA	3	2	4
ABCD	3	2	4

P has the loop A-B-C-A and fixed D. Q swaps A/B and C/D. R has one four-letter loop. Under P the full rows are ABCD, BCAD, CABD, ABCD. A fixed D alone does not certify that the whole key has returned. For Problem 3, choose output rows **BCAD** and **BCDA**. Both send ABBA to BCCB, but ABCD returns in three and four turns respectively. The missing letters C and D leave room for different keys. If needed, ask which key entries the message ABBA actually tests. Extension: can two distinct keys agree on a message containing every letter? No: that message tests every input.

Middle 4-5: construction and completeness

The possible first returns for ABCD are **1,2,3,4**. Example output rows: ABCD, BACD, BCAD, BCDA. A message containing every letter returns exactly when the whole key does. Four letters have these five possible loop-size lists:

Sizes	4	3+1	2+2	2+1+1	1+1+1+1
Return	4	3	2	2	1

No list gives six. Adding E allows A-B-C-A and D-E-D, output **BCAED**; the first simultaneous return is six. Merely testing many keys does not prove that four letters cannot work. Use the loop argument below and exhaust the five size lists if the child wants a proof. Do not supply these lists before the construction attempts.

Why loops and common returns settle the question

Follow a symbol until one repeats. The first repeat must be the start: a later first repeat would give a symbol two predecessors, contradicting reversibility. Remove that loop and continue with unused symbols. This splits the key into disjoint loops, including fixed letters as one-loops. A loop returns at multiples of its length; the whole key returns at the first common multiple of all lengths. If needed, move counters around a three-loop and a two-loop and mark home arrivals. "Least common multiple" names this result; it is not an entry requirement.

Five letters and key order

Upper 1–2: a maximum without a prescribed method

The printed R,S,T,U keys first return after **5,6,4,2** turns. S is slowest among these. Its full trace is ABCDE, BCAED, CABDE, ABCED, BCADE, CABED, ABCDE. R is one five-loop; S has separate loops of sizes three and two; T has sizes four and one; U has sizes two, two, and one. For Problem 2, possible returns are **1,2,3,4,5,6**. Witness output rows are ABCDE, BACDE, BCADE, BCDAE, BCDEA, BCAED respectively. The complete loop-size classification is:

Sizes	5	4+1	3+2	3+1+1	2+2+1	2+1+1+1	1+1+1+1+1
Return	5	4	6	3	2	2	1

To prove completeness, use the loop argument on page 3. A largest loop of five or four leaves only one split; three leaves two or one+one; two leaves two+one or three ones; one forces all singletons. Therefore six is the maximum, and no time is missing from the listed possibilities. **Hints, only if needed:** follow one letter; ask what happens to unused letters; ask how to organize the keys already found. If checking all keys becomes copying, draw just their loops. A child may find another valid classification. Blank workspace deliberately replaces the former largest-loop table.

Upper 3: overlapping keys need a real test

P swaps A/B; Q swaps B/C; L swaps D/E. V and W are opposite three-cycles on A,B,C, fixing D,E.

Pair	First then second	Second then first
P,Q	CABDE	BCADE
P,L	BACED	BACED
V,W	ABCDE	ABCDE

P and Q do not commute: A goes through B to C in one order, and through A to B in the other. P and L commute because they affect disjoint sets of letters. V and W *overlap* but commute: either order undoes the three-cycle. Thus “overlap always changes the answer” is false. An exact rule is: reversing the order leaves every message unchanged *between the two orders* precisely when the two routes give the same final letter for *each* input letter. Checking ABCDE tests every input. Disjoint action is sufficient, not necessary. Children may reach a sufficient rule before an exact one; distinguish their scope. There is no requirement to classify all commuting permutations. **If needed:** keep the intermediate message visible; test one disputed letter. **Extension:** invent another overlapping pair that does commute (a key and its inverse, or two powers of the same key), and one that does not.

Upper 4: the combined machine and its inverse

P followed by Q sends ABCDE to CABDE. Repeating this whole two-key turn gives BCADE and then ABCDE, so its first return is **three machine turns**, or six individual key applications. The combined key has the loop A–C–B–A and fixes D,E. To undo one P-then-Q turn, use **Q then P**. Each swap undoes itself, and the inverse reverses their order. From CABDE, Q gives BACDE and P restores ABCDE. Every input letter is tested, so the same inverse works for any message. Optional extension: a general two-key machine is undone by the second key’s inverse, then the first key’s inverse.

Long returns and complete bounds

Extra 1: construct before optimizing

With A–H, suitable loop sizes for returns 4,6,8,12,15 are respectively **4+4, 6+2, 8, 4+3+1, 5+3**. Put successive letters around each loop to obtain actual keys. These are examples, not unique answers. Their output rows can be BCDAFGHE, BCDEFAHG, BCDEFGHA, BCDAFGEH, and BCDEAGHF. Check the 12-turn example as A–B–C–D–A, E–F–G–E, H fixed. If needed, first build one loop and then two independent loops. Two four-loops return together after four turns, not sixteen; the return is a common multiple, not automatically the product. Let children compare attempts before naming LCM. Extension: find other loop-size lists giving the same return.

Extra 2: records for eight and nine letters

Eight letters achieve **15** with 5+3; nine achieve **20** with 5+4. The proof needs both a construction and a bound. Following the loop theorem, classify by the largest loop. These are adult solutions, not a template to hand out at the start.

Letters	Largest loop	Greatest return in that case
8	8, 7, 6	8, 7, 6
	5	15 (5+3)
	4	12 (4+3+1)
	at most 3	6
9	9, 8, 7, 6	9, 8, 14, 6
	5	20 (5+4)
	4	12 (4+3+2)
	at most 3	6

For eight letters with largest loop five, the remaining three split as 3,2+1,1+1+1, giving returns 15,10,5. With largest four, the leftover four split as 4,3+1,2+2,2+1+1,1+1+1+1, giving 4,12,4,4,4. Larger loops leave only one or two letters, giving the displayed bounds. If every loop has size at most three, every length divides six, so the return is at most six. For nine letters with largest five, the remaining four give returns 20,15,10,10,5. With largest four, all remaining loops have lengths 1–4, so their common return divides twelve; 4+3+2 attains twelve. With largest seven, the leftover two give at most fourteen; with largest six, all leftover lengths divide six. The cases cover every key, proving both optima.

Extra 3: an extra letter need not improve the record

For sizes 1 through 9 the maxima are **1,2,3,4,6,6,12,15,20**. The five-to-six-letter plateau disproves strict growth. For six letters, largest loops 6,5,4,3,2,1 give case maxima 6,5,4,6,2,1: for largest three the leftover three can supply a two-loop, attaining six. For seven letters, largest loops 7,6,5,4, at most 3 give maxima 7,6,10,12,6; 4+3 attains twelve. Sizes up to five follow by the earlier partition arguments (sizes one to three have maxima one, two, three). The record can *never decrease*: keep an optimal old key and make the new letter fixed. Its return time is unchanged. This proves monotonicity for every size, beyond the nine checked values. Do not describe nine examples alone as proof of a universal statement.

Sources, continuity, and next evidence

What changed, and why

The organizer approved the concise September 30 Week 2 catalogs: short prompts can support sustained construction, impossibility, optimality, and completeness problems. Week 3 now groups contrasting cases under substantial questions. Shared rules appear once; letter tables specify keys or leave room for candidates. Blank work areas allow children to choose how to record. Tracing every turn, drawing loops, sorting by largest loop, and marking common multiples are available adult supports rather than printed procedures. The extra packet retains proof as its explicit task. Earlier pages also retain central impossibility and completeness questions. Concision removes repeated instructions, not mathematical depth. These changes are design proposals; no classroom result for this revision is claimed.

Source passages reviewed locally

Natasha Rozhkovskaya, *Math Circles for Elementary School Students*: Lesson 2, Problems 2.2–2.8 (EPUB OEBPS/part0012.xhtml) supplies coded messages and repeated-letter clues. Its lesson report describes strong reasoning by children who were not yet fluent readers. Our short letter-label clues and arbitrary reversible keys adapt that entry; we are not reproducing the shift-code worksheet. Lesson 3, “At the lesson,” item 1 (part0013.xhtml), describes letting children attempt and explain before introducing a table. Lesson 8, item 2 (part0018.xhtml), reports writing and coloring pace separating groups. Our choice of optional adult records and generous unstructured workspace responds to these accounts; the sources do not establish that our particular pages will work. ***Math Circle by the Bay, preface, printed pp. viii–x (PDF pp. 9–11)***: substantial themes, manipulatives, clear statements, independent solving, reserve challenges, and different pace and depth across ages. Our exact prompts, timing, staffing, and five-minute-plus design aim are applications, not prescriptions quoted from the book. **Lowell year 1, Handout 8, Problem 8.1**: divisibility in sharing apples among possible guest counts provides continuity with common returns. The saved handout does not establish which children used it. This week changes the representation and question to reversible machines and maximum return time. Week 4 then restricts arbitrary keys to fixed shifts around a circle.

Mathematical lineage

The existing design relates these tasks to permutations, disjoint cycles, inverses, and products (Judson, *Abstract Algebra: Theory and Applications*, Chapter 5, §5.1). The explicit loop proof in this guide is self-contained. Maximal permutation order is Landau’s function: the exact finite question in the extra packet. The prior design’s reference is Deléglise, Nicolas, Zimmermann, *Landau’s function for one million billions*, 2008 manuscript, §1.1–1.3. We retain that lineage; this revision makes no claim about current research status. The small values here are established by the displayed finite arguments and separately verified by enumeration.

Record only what happened

Use edition IDs F03-K/M/U/X-v4 and the actual problem, key, and starting message in the use log. Record whether children changed each old symbol exactly once, distinguished message return from whole-key return, invented a useful record, found counterexamples, or needed repeated adult rescue. Save interesting invented keys. Mark a result as tried, conjectured, checked in cases, proved, or supplied. Keep observed actions separate from explanations and proposed fixes. Record what children wanted to continue. On a future return, use new alphabets, composition questions, or larger-size records instead of automatically repeating these same keys. Unused pages remain reserve material.